

Homework 1. Molotov Vladimir

Problem 1. Pareto distribution

$$f(x) = \begin{cases} \frac{\alpha x_m^\alpha}{x^{\alpha+1}}, & x \geq x_m \\ 0, & x < x_m \end{cases}$$

① Moments:

$$E[X] = \frac{\alpha x_m}{\alpha - 1}, \quad \alpha > 1$$

$$E[X^2] = \frac{\alpha x_m^2}{\alpha - 2}, \quad \alpha > 2$$

Sample: $\bar{X} = \frac{1}{n} \sum_{i=1}^n x_i$

$$\bar{X}^2 = \frac{1}{n} \sum_{i=1}^n x_i^2$$

$$\bullet x_m = \bar{X} \frac{\alpha - 1}{\alpha}$$

$$\bar{X}^2 = \frac{\alpha x_m^2}{\alpha - 2} = \frac{\alpha \cdot (\bar{X})^2 (\alpha - 1)^2}{\alpha^2 (\alpha - 2)}$$

$$\left(\frac{\bar{X}^2}{\bar{X}^2} \right)^{\stackrel{!}{=t}} = \frac{(\alpha - 1)^2}{\alpha (\alpha - 2)}$$

$$t \alpha (\alpha - 2) = (\alpha - 1)^2$$

$$t \alpha^2 - 2t \alpha = \alpha^2 - 2\alpha + 1$$

$$(t - 1) \alpha^2 + (2 - 2t) \alpha - 1 = 0$$

$$(t - 1) \alpha^2 - 2(t - 1) \alpha - 1 = 0$$

~~Q.E.D.~~ ~~Q.E.D.~~

$$k := t - 1 = \frac{\bar{X}^2}{\bar{X}^2} - 1$$

$$k \alpha^2 - 2k \alpha - 1 = 0$$

$$\alpha = \frac{2k + \sqrt{4k(k+1)}}{2k} = 1 + \sqrt{\frac{k+1}{k}} = 1 + \sqrt{1 - \frac{1}{t}}$$

$$\hat{L}_{mom} = 1 + \frac{1}{\sqrt{1 - \frac{\overline{X}^2}{\overline{X^2}}}} = 1 + \frac{1}{\sqrt{1 - \frac{\overline{X}^2}{\overline{X^2}}}}$$

$$\hat{x}_{mom} = \overline{X} \cdot \frac{\hat{L}_{mom} - 1}{\hat{L}_{mom}}, d \geq 2$$

~~when d=2~~ →

$$d \rightarrow 2: \hat{L}_{mom} = 2$$

$$\hat{x}_{mom} = 2$$

Homework 1

May 25, 2025

Problem 1. Consider a sample $X_n = \{x_1, \dots, x_n\}$ of i.i.d. observations from Pareto distribution with PDF:

$$f_X(x) = \begin{cases} \frac{\alpha x_m^\alpha}{x^{\alpha+1}} & x \geq x_m \\ 0 & \text{otherwise} \end{cases}$$

with (unknown) parameters $x_m > 0$ (called scale), and $\alpha > 0$ (called shape). Denote $\theta = (\alpha, x_m)$.

```
[1]: import numpy as np
import matplotlib.pyplot as plt
import statsmodels.api as sm
```

```
[2]: alpha_true = 5
xm_true = 2
n_list = [100, 200, 500, 1000, 2000]
```

Task 2. QQ-plots

```
[3]: for n in n_list:
    bootstrap_alpha = []
    xm_list = []

    for _ in range(1_000):
        data = xm_true * np.random.pareto(alpha_true, size=n)

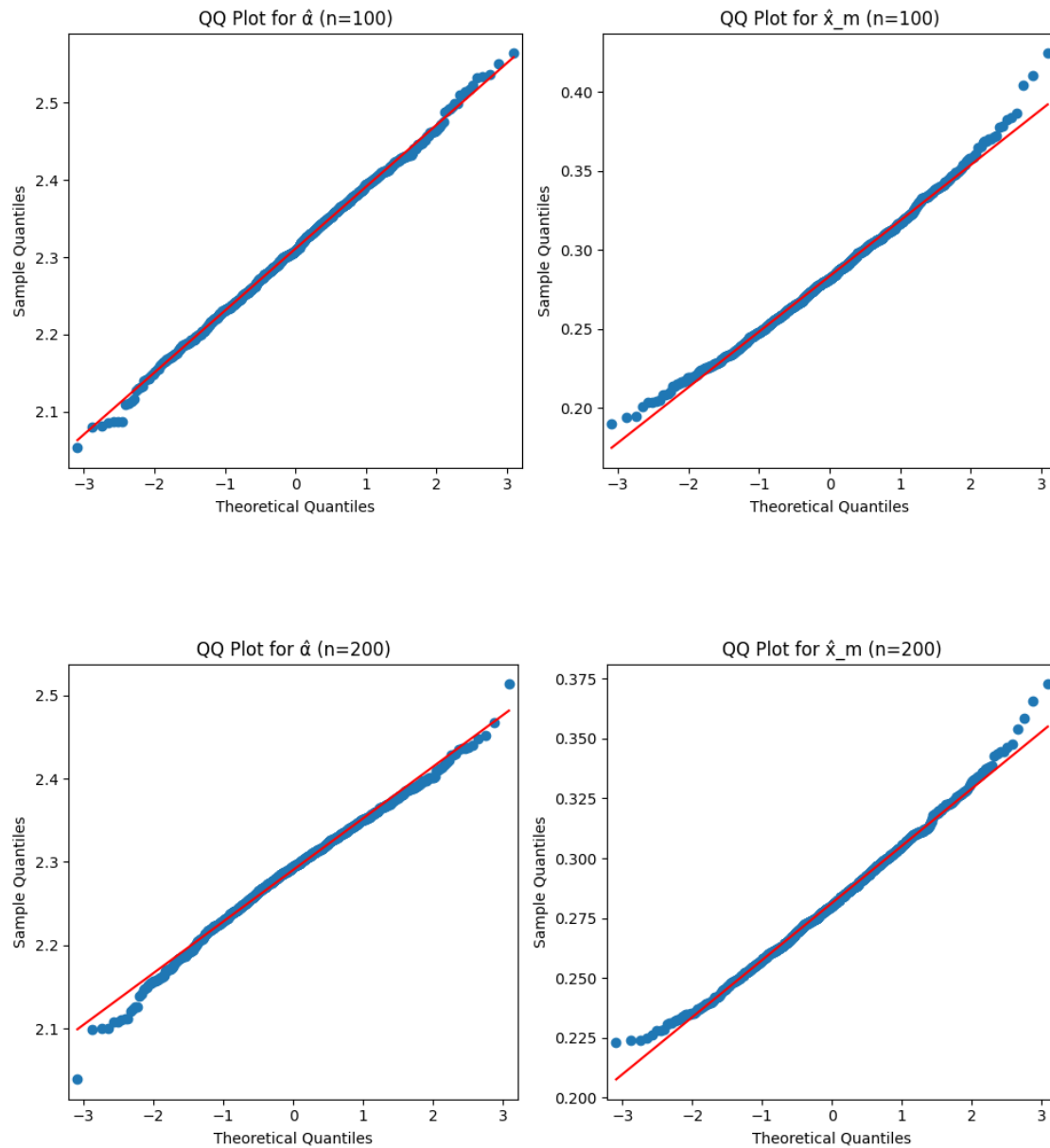
        sample_mean = np.mean(data)
        sample_second_moment = np.mean(data ** 2)

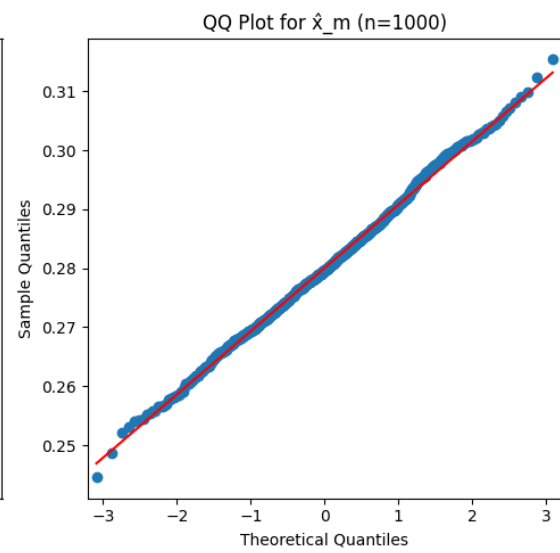
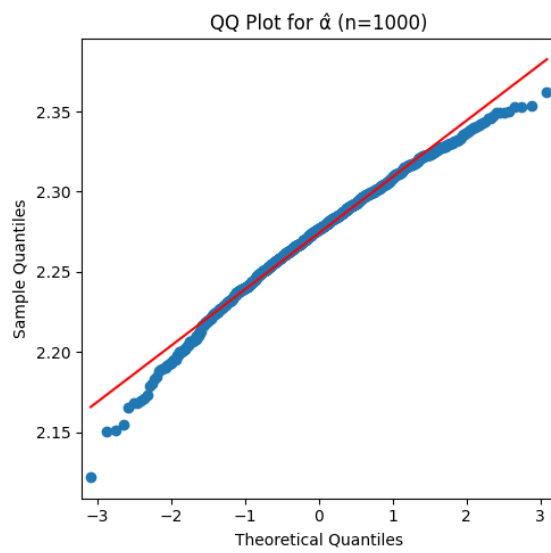
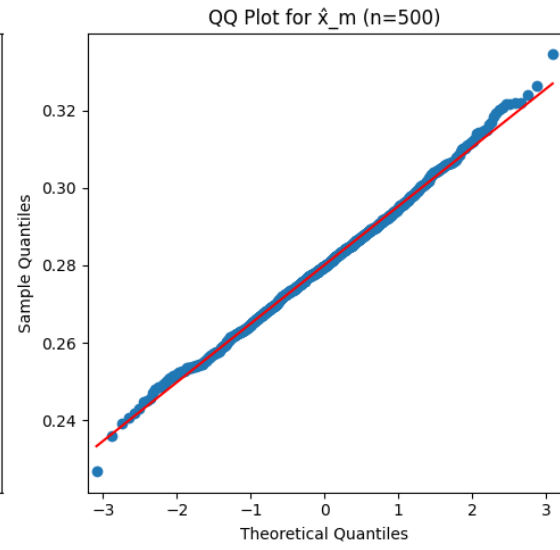
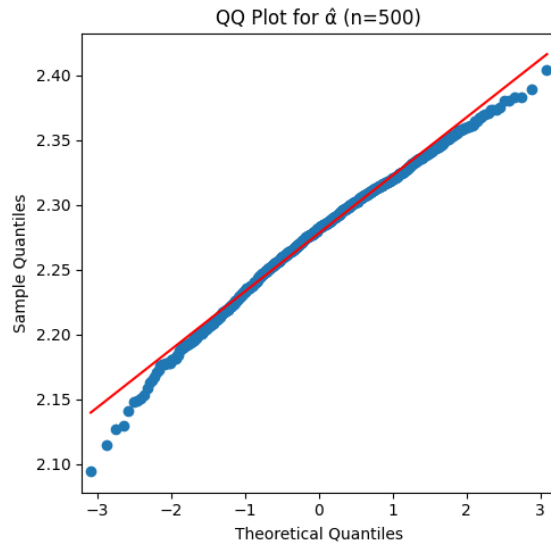
        k = (sample_mean ** 2) / sample_second_moment
        alpha = 1 + 1 / np.sqrt(1 - k)
        xm = sample_mean * (alpha - 1) / alpha

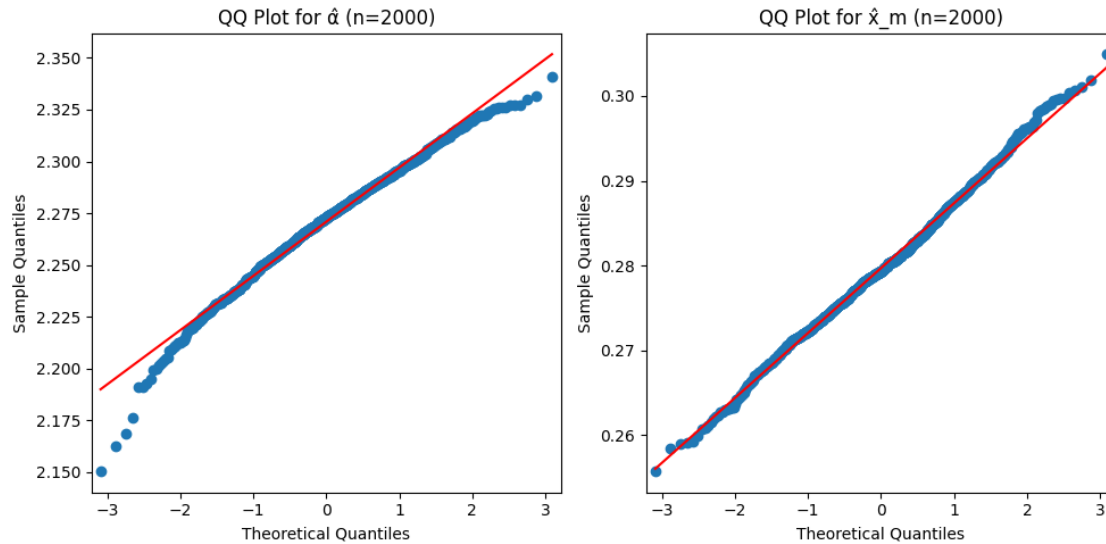
        bootstrap_alpha.append(alpha)
        xm_list.append(xm)

    fig, axes = plt.subplots(1, 2, figsize=(10, 5))
    sm.qqplot(np.array(bootstrap_alpha), line='s', ax=axes[0])
    axes[0].set_title(f'QQ Plot for ^ (n={n})')
```

```
sm.qqplot(np.array(xm_list), line='s', ax=axes[1])
axes[1].set_title(f'QQ Plot for  $\hat{x}_m$  (n={n})')
plt.tight_layout()
plt.show()
```







Task 3. Bootstrap

```
[4]: avg_var_alpha_list = []
avg_var_xm_list = []

for n in n_list:
    current_var_alpha = []
    current_var_xm = []

    for _ in range(100):
        for _ in range(1_000):
            data = xm_true * np.random.pareto(alpha_true, size=n)
            sample_mean = np.mean(data)
            sample_second_moment = np.mean(data ** 2)
            k = (sample_mean ** 2) / sample_second_moment

            # Bootstrap resampling
            bootstrap_alpha = []
            bootstrap_xm = []

            for _ in range(1000):
                bootstrap_sample = np.random.choice(data, size=n, replace=True)
                bootstrap_mean = np.mean(bootstrap_sample)
                boot_second_moment = np.mean(bootstrap_sample ** 2)
                k_boot = (bootstrap_mean ** 2) / boot_second_moment

                alpha_hat = 1 + 1 / np.sqrt(1 - k_boot)
                xm_hat = bootstrap_mean * (alpha_hat - 1) / alpha_hat
                bootstrap_alpha.append(alpha_hat)
```

```

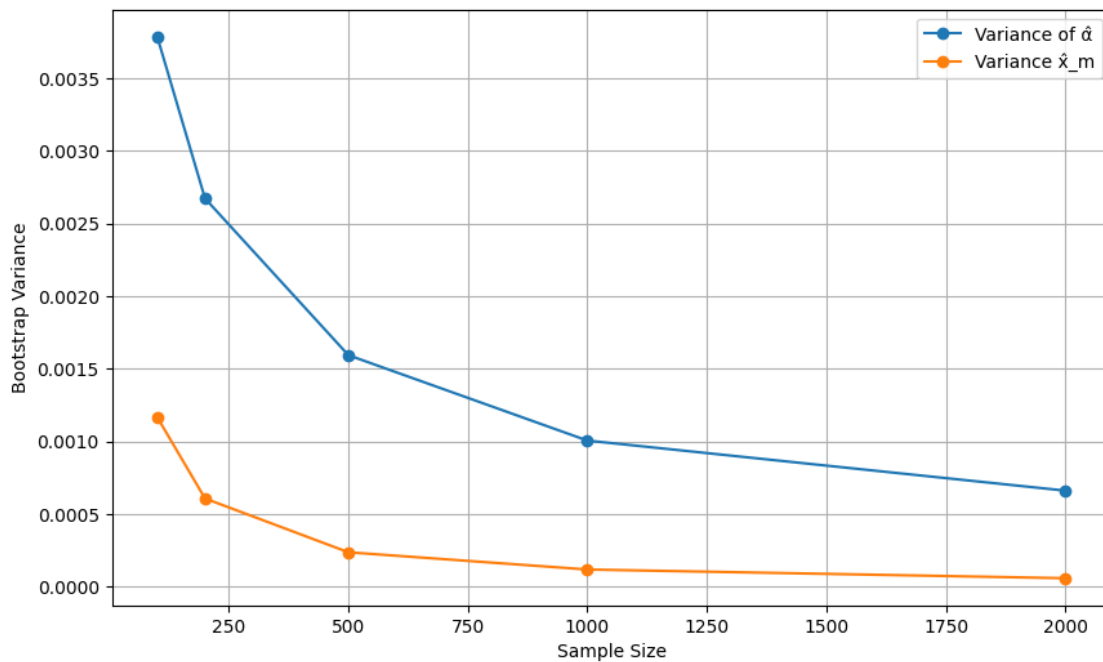
bootstrap_xm.append(xm_hat)

current_var_alpha.append(np.var(bootstrap_alpha, ddof=1))
current_var_xm.append(np.var(bootstrap_xm, ddof=1))

avg_var_alpha_list.append(np.mean(current_var_alpha))
avg_var_xm_list.append(np.mean(current_var_xm))

plt.figure(figsize=(10, 6))
plt.plot(n_list, avg_var_alpha_list, 'o-', label='Variance of  $\hat{\alpha}$ ')
plt.plot(n_list, avg_var_xm_list, 'o-', label='Variance  $\hat{x}_m$ ')
plt.xlabel('Sample Size')
plt.ylabel('Bootstrap Variance')
plt.legend()
plt.grid(True)
plt.show()

```



Task 4.

```

[5]: def fisher_information_matrix(alpha, xm, n):
    I11 = n / alpha**2
    I12 = -n / xm
    I22 = n * alpha / xm**2
    return f"{I11}    {I12}\n{n}{I12} {I22}"

```

```

n = 100
print("Fisher Information Matrix:")
print(fisher_information_matrix(alpha_true, xm_true, n))

```

Fisher Information Matrix:

```

4.0   -50.0
-50.0 125.0

```

```

[6]: def count_crlb_alpha(n, alpha, xm):
      return (alpha ** 3) / (n * (alpha - 1))

      def count_crlb_xm(n, alpha, xm):
          return (xm ** 2) / (n * (alpha - 1))

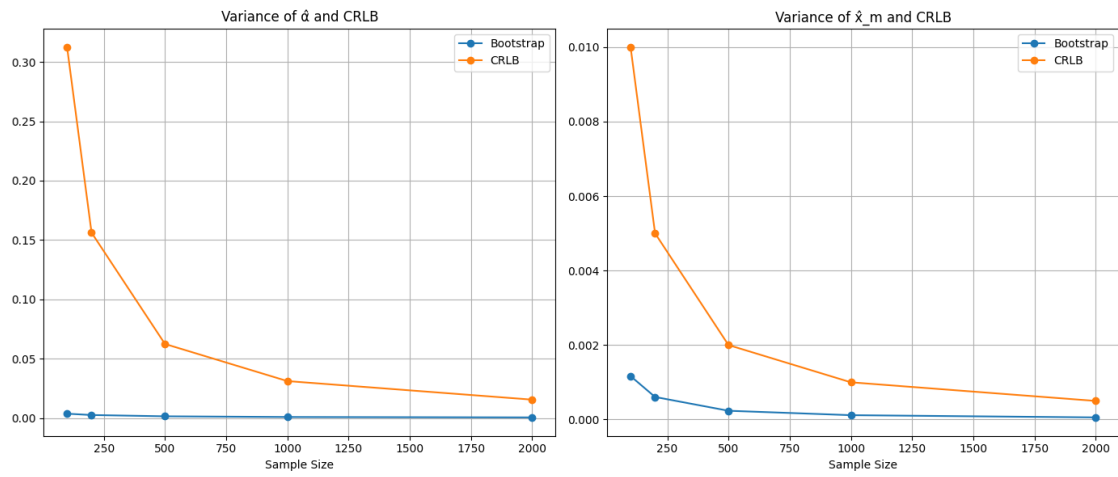
      crlb_alpha = [count_crlb_alpha(n, alpha_true, xm_true) for n in n_list]
      crlb_xm = [count_crlb_xm(n, alpha_true, xm_true) for n in n_list]

      plt.figure(figsize=(14, 6))
      plt.subplot(1, 2, 1)
      plt.plot(n_list, avg_var_alpha_list, 'o-', label='Bootstrap')
      plt.plot(n_list, crlb_alpha, 'o-', label='CRLB')
      plt.xlabel('Sample Size')
      plt.title('Variance of  $\hat{\alpha}$  and CRLB')
      plt.legend()
      plt.grid(True)

      plt.subplot(1, 2, 2)
      plt.plot(n_list, avg_var_xm_list, 'o-', label='Bootstrap')
      plt.plot(n_list, crlb_xm, 'o-', label='CRLB')
      plt.xlabel('Sample Size')
      plt.title('Variance of  $\hat{x}_m$  and CRLB')
      plt.legend()
      plt.grid(True)

      plt.tight_layout()
      plt.show()

```



Task 5. Maximum likelihood estimate.

$$L(d, x_m) = \prod_{i=1}^n \frac{d x_m^d}{x_i^{d+1}}, \quad x_i \geq x_m$$

$$\hat{x}_m = \min\{x_1, \dots, x_n\}$$

$$\log L(d) = n \log d + n d \log \hat{x}_m - (d+1) \sum_{i=1}^n \log x_i$$

$$\frac{\partial \log L(d)}{\partial d} = 0$$

$$\frac{\partial \log L(d)}{\partial d} = \frac{n}{d} + n \log \hat{x}_m - \sum_{i=1}^n \log x_i = 0$$

$$\frac{n}{d} = -n \log \hat{x}_m + \sum_{i=1}^n \log x_i$$

$$\hat{d} = \frac{n}{\sum_{i=1}^n \log x_i - n \log \hat{x}_m}$$

$$\hat{d} = \frac{n}{\sum_{i=1}^n \log \left(\frac{x_i}{\hat{x}_m} \right)}$$

Problem 2. Gamma distribution.

$$f(x) = \frac{1}{\Gamma(k) \theta^k} x^{k-1} e^{-x/\theta}, \quad x > 0$$
$$k > 0$$
$$\theta > 0$$

Task 1.

~~write down the pdf~~

• fixed k :

$$\log f(x) = -\log \Gamma(k) - k \log \theta + (k-1) \log x - \frac{x}{\theta}$$

~~write down the log-likelihood~~

$$f(x) = x^{k-1} \cdot \frac{1}{\Gamma(k)} \cdot \exp\left(-\frac{x}{\theta} - k \log \theta\right)$$

$$p(x) = \exp\left(\sum_i \eta_i(\theta) T_i(x) - A(\eta)\right)$$

$$h(x) = \frac{x^{k-1}}{\Gamma(k)}; \quad T(x) = x; \quad \eta(\theta) = -\frac{1}{\theta}; \quad A(\eta) = -k \log(-\eta)$$

$$f(x) = h(x) \cdot \exp(\eta(\theta) \cdot x - A(\eta))$$

Natural parameter: $\eta = -\frac{1}{\theta}$

Sufficient statistic: $T(x) = x$

$$\frac{\partial A(\eta)}{\partial \eta} = \frac{k}{\eta} \Rightarrow E[X] = k\theta$$

$$\frac{\partial^2 A(\eta)}{\partial \eta^2} = -\frac{k}{\eta^2} \Rightarrow \text{Var}[X] = k\theta^2$$

Task 2.

$$\bullet \text{ } L(\theta) = \prod_{i=1}^n f(x_i) = \left(\frac{1}{\Gamma(k)\theta^k} \right)^n \cdot \prod_{i=1}^n x_i^{k-1} \exp(-x_i/\theta)$$

$$\log L(\theta) = -n \log \Gamma(k) - nk \log \theta + (k-1) \sum_{i=1}^n \log x_i - \frac{1}{\theta} \sum_{i=1}^n x_i$$

$$\frac{\partial \log L(\theta)}{\partial \theta} = -\frac{nk}{\theta} + \frac{1}{\theta^2} \sum_{i=1}^n x_i = 0$$

$$+ nk = \sum_{i=1}^n x_i$$

$$\theta = \frac{1}{nk} \sum_{i=1}^n x_i$$

$$\hat{\theta} = \frac{\bar{X}}{k}$$

$$\bullet E[\hat{\theta}] = \frac{1}{k} E[\bar{X}] = \frac{1}{k} \cdot \frac{1}{n} \sum_{i=1}^n E(x_i) = \frac{1}{k} \cdot \frac{1}{n} \cdot nk\theta = \theta$$

$$E[\hat{\theta}] = \theta \Rightarrow \text{unbiased}$$

Task 3. Fisher information matrix.

$$H_{ij} = \frac{\partial^2 \ell_n}{\partial \theta_i \partial \theta_j}$$

$$I_{k\theta} = -\frac{n}{\theta}, \quad I_{\theta\theta} = \frac{nk}{\theta^2}$$

$I_{k\theta} \neq 0 \Rightarrow k$ and θ are not orthogonal

$$\text{Var}(\hat{\theta}) \geq \frac{1}{I_{\theta\theta}} = \frac{\theta^2}{nk}$$

$$se = \frac{\hat{\theta}}{\sqrt{nk}}$$

Task 4. Delta method.

$$\cdot L=k, \beta = \frac{1}{\theta} \Rightarrow f(x) = \frac{\beta^k}{\Gamma(k)} x^{k-1} e^{-\beta x}, x > 0$$

$$\hat{\beta}_{MLE} = \frac{1}{\hat{\theta}_{MLE}} = \frac{k}{\bar{x}}$$

• Delta method

$$\text{Var}(\hat{\beta}) \approx [g'(\theta)]^2 \cdot \text{Var}(\theta)$$

$$g(\theta) = \frac{1}{\theta}, \quad g'(\theta) = -\frac{1}{\theta^2}$$

$$\text{Var}(\theta) = \frac{\theta^2}{nk} \Rightarrow \text{Var}(\hat{\beta}) = \left(-\frac{1}{\theta^2}\right)^2 \cdot \frac{\theta^2}{nk} = \frac{1}{\theta^2 nk} = \frac{\beta^2}{nk}$$

$$\boxed{se(\hat{\beta}) = \frac{\beta}{\sqrt{nk}}}$$

Homework 2

May 25, 2025

0.0.1 Problem 1.

```
[1]: from math import comb
import numpy as np
from scipy.stats import norm
np.random.seed(42)
```

```
[2]: T = [0.225, 0.262, 0.217, 0.240, 0.230, 0.229, 0.235, 0.217]
S = [0.209, 0.205, 0.196, 0.210, 0.202, 0.207, 0.224, 0.223, 0.220, 0.201]

mean_T = np.mean(T)
mean_S = np.mean(S)
var_T = np.var(T, ddof=1)
var_S = np.var(S, ddof=1)
n_T = len(T)
n_S = len(S)

se_diff = np.sqrt(var_T/n_T + var_S/n_S)
Z = (mean_T - mean_S) / se_diff
p_value = 2 * (1 - norm.cdf(abs(Z)))
Z_crit = norm.ppf(0.975)

ci_low = (mean_T - mean_S) - Z_crit * se_diff
ci_high = (mean_T - mean_S) + Z_crit * se_diff

print(f"p-value: {p_value:.4f} < 0.05 -> The average proportion of words in_
↳Twain's essays is different from that of Snodgrass.")
print(f"95% confidence interval: ({ci_low:.4f}, {ci_high:.4f})")
```

p-value: 0.0002 < 0.05 -> The average proportion of words in Twain's essays is different from that of Snodgrass.
95% confidence interval: (0.0104, 0.0339)

```
[3]: combined = np.array(T + S)
perm_diffs = []

for _ in range(10_000):
    np.random.shuffle(combined)
    perm_T = combined[:n_T]
```

```

perm_S = combined[n_T:]
perm_diffs.append(np.mean(perm_T) - np.mean(perm_S))

p_value = np.mean(np.abs(perm_diffs) >= abs(mean_T - mean_S))

print(f"P-value: {p_value:.4f} < 0.05")

```

P-value: 0.0008 < 0.05

0.0.2 Problem 2.

```

[4]: placebo = (80, 45)
p_values = []
drugs = {
    "Chlorpromazine": (75, 26),
    "Dimenhydrinate": (85, 52),
    "Pentobarbital (100 mg)": (67, 35),
    "Pentobarbital (150 mg)": (85, 37)
}

def fisher_exact_test(a, b, c=45, d=35):
    n = a + b + c + d
    row1 = a + b
    row2 = c + d
    col1 = a + c
    col2 = b + d

    def hypergeom_prob(x):
        return comb(col1, x) * comb(col2, row1 - x) / comb(n, row1)

    p_observed = hypergeom_prob(a)
    p_value = 0.0
    for i in range(max(0, row1 - col2), min(row1, col1) + 1):
        p = hypergeom_prob(i)
        if p <= p_observed:
            p_value += p

    odds_ratio = (a * d) / (b * c)
    return odds_ratio, p_value

print(f"{'Drug':37} {'Odds Ratio':12} {'P-value':12}")
for drug, (total, a) in drugs.items():
    b = total - a
    odds_ratio, p_value = fisher_exact_test(a, b)
    p_values.append(p_value)
    print(f"{drug:30} {odds_ratio:12.3f} {p_value:12.3f}")

```

Drug	Odds Ratio	P-value
Chlorpromazine	0.413	0.010
Dimenhydrinate	1.226	0.531
Pentobarbital (100 mg)	0.851	0.740
Pentobarbital (150 mg)	0.600	0.120